

Totalne kolorowanie grafu - Raport

Klaudia Buczek, Klara Pluta, Gabriela Warchoł

1 Opis problemu

Totalne kolorowanie grafu to problem polegający na przypisaniu kolorów zarówno wierzchołkom, jak i krawędziom grafu, tak aby żadne dwa sąsiednie elementy (wierzchołki lub krawędzie) nie miały tego samego koloru. Formalnie, totalne kolorowanie grafu możemy opisać przez funkcję: $f : V \cup E \rightarrow \{1, 2, \dots, k\}$, gdzie V to zbiór wierzchołków, E to zbiór krawędzi, a k to liczba dostępnych kolorów. Ponadto muszą zachodzić następujące warunki:

- Dla każdej krawędzi $uv \in E$ zachodzi $f(u) \neq f(v)$.
- Dla każdych dwóch różnych krawędzi $uv, ux \in E$ zachodzi $f(uv) \neq f(ux)$.
- Dla każdej krawędzi $uv \in E$ zachodzi $f(u) \neq f(uv)$ oraz $f(v) \neq f(uv)$.

Celem projektu jest wykorzystanie SAT-solverów do zbadania, czy dla zadanego grafu G oraz parametru $k \in \mathbb{N}$ istnieje totalne kolorowanie.

2 Modelowanie formuły logicznej

Aby rozwiązać problem totalnego kolorowania grafu za pomocą SAT-solvera, musimy przekształcić go w formułę logiczną. W tym celu wprowadzamy zmienne logiczne, które reprezentują przypisanie kolorów do wierzchołków i krawędzi grafu.

2.1 Zmienne logiczne

Definiujemy zmienną $x_{a,b}$ dla każdego elementu $a \in V \cup E$ oraz każdego koloru $b \in K$ (gdzie K to zbiór dostępnych kolorów). Zmienna $x_{a,b}$ przyjmuje wartość 1, jeśli element a jest pokolorowany kolorem b , i 0 w przeciwnym przypadku.

2.2 Ograniczenia

Aby zapewnić poprawność totalnego kolorowania, musimy wprowadzić odpowiednie ograniczenia:

- Każdy element (wierzchołek lub krawędź) musi być pokolorowany dokładnie jednym kolorem. To ograniczenie dzielimy na dwie części:
 - Przynajmniej jeden kolor: $\forall a \in V \cup E \quad (x_{a,1} \vee x_{a,2} \vee \dots \vee x_{a,k})$
 - Co najwyżej jeden kolor: $\forall a \in V \cup E \quad \forall b, c \in K, b < c \quad (\neg x_{a,b} \vee \neg x_{a,c})$

- Sąsiednie wierzchołki nie mogą mieć tego samego koloru:

$$\forall uv \in E, b \in K \quad (\neg x_{u,b} \vee \neg x_{v,b})$$

- Krawędzie wychodzące z jednego wierzchołka muszą mieć różne kolory:

$$\forall e_1, e_2 \in E : e_1 \cap e_2 \neq \emptyset \quad \forall b \in K \quad (\neg x_{e_1,b} \vee \neg x_{e_2,b})$$

- Krawędź i jej końce nie mogą mieć tego samego koloru:

$$\forall uv \in E, b \in K \quad (\neg x_{u,b} \vee \neg x_{uv,b}) \wedge (\neg x_{v,b} \vee \neg x_{uv,b})$$

3 Implementacja

Program został podzielony na kilka modułów, z których każdy odpowiada za inną część procesu rozwiązywania problemu totalnego kolorowania grafu.

- `total_coloring.py` - Główny program, wczytuje plik z grafem, zmienia krawędzie, wierzchołki i kolory na zmienne logiczne oraz tworzy formułę logiczną w formacie DIMACS.
- `run_experiments.py` - Skrypt do automatycznego uruchamiania testów. Bierze po kolei grafy z folderu `examples` i sprawdza je dla coraz większej liczby kolorów, zapisując wyniki w pliku `results.txt`. Skrypt zatrzymuje się od razu, gdy solver zwróci pierwszy wynik SAT, dzięki temu wiemy, jaka jest minimalna liczba kolorów dla danego grafu.
- `verify_unsat.py` - Kod do wyciągania dowodów niespełnialności za pomocą solvera CaDiCaL.
- `check_cnf.py` - Skrypt do sprawdzenia jednego pliku CNF przez solver Glucose3.

3.1 Algorytm działania pętli testowej

Działanie skryptu uruchamiającego eksperymenty pokazuje Algorytm 1.

Algorytm 1 Automatyczne testowanie grafów

Require: Folder z grafami **Examples**, zakres kolorów $K_VALUES = [2, \dots, 13]$

Ensure: Plik z wynikami `results.csv`

```

1: for graf  $g \in \text{Examples}$  do
2:   Wczytaj wierzchołki  $V$  i krawędzie  $E$ 
3:   for  $k \in K\_VALUES$  do
4:     Wygeneruj formułę CNF dla parametrów  $(G, k)$ 
5:     Uruchom SAT-solver
6:     Zapisz wyniki (liczbę zmiennych, klauzul i wynik SAT/UNSAT)
7:     if  $result == \text{"SAT"}$  then
8:       break ▷ Mamy wynik, nie liczymy dla większych  $k$ 
9:     end if
10:  end for
11: end for
```

4 Wyniki eksperymentów

Przetestowaliśmy nasz model na wielu różnych rodzajach grafów (ścieżki, cykle, grafy pełne, dwudzielne, koła, gwiazdy oraz graf Petersena).

4.1 Minimalna liczba kolorów

W tabeli 1 widać, przy jakiej najmniejszej liczbie kolorów k solver był w stanie znaleźć poprawne totalne kolorowanie (czyli dał wynik SAT).

Tabela 1: Wyniki minimalnego k dla badanych grafów

Graf	Minimalne k	Graf	Minimalne k
P4	3	C10	4
P6	3	diamond	4
P8	3	Q3	4
P10	3	K3,3	5
C6	3	K4	5
C4	4	K5	5
C8	4	K4,4	6
W5	6	Petersen	6
S5	6	K5,5	7
W7	8	K6	7
S8	9		

Wszystkie wyniki zgadzają się z teorią grafów. Im graf ma więcej krawędzi i im więcej z nich schodzi się w jednym wierzchołku, tym więcej kolorów potrzebuje solver, żeby spełnić wszystkie warunki.

4.2 Rozmiar wygenerowanych plików CNF

Liczba zmiennych w formule zawsze zależy tylko od rozmiaru grafu i liczby kolorów, co idealnie potwierdza wzór:

$$\text{Liczba zmiennych} = (|V| + |E|) \cdot k$$

Dużo ciekawsza jest liczba klauzul, czyli ograniczeń. Jest ona uzależniona od stopnia wierzchołków. Tabela 2 pokazuje największe formuły uzyskane w testach.

Tabela 2: Największe formuły uzyskane w testach

Graf	k	Zmienne	Klauzule	Wynik
K5,5	7	245	1995	SAT
W7	8	176	1310	SAT
K6	7	147	1197	SAT
S8	9	153	1097	SAT

Gdy graf ma dużo krawędzi, liczba klauzul drastycznie rośnie. Dla prostej ścieżki P_{10} program wygenerował tylko 181 klauzul, a dla grafu pełnego K_6 o podobnej liczbie elementów (wierzchołków i krawędzi) wyszło ich aż 1197.

4.3 Weryfikacja przypadków UNSAT i DRAT

W projekcie spróbowaaliśmy uruchomić pełną weryfikację za pomocą programu **drat-trim**. Udało nam się go skompilować i przygotować całe środowisko. Pojawił się jednak problem techniczny: biblioteka **PySAT**, której używamy w Pythonie do obsługi solverów, nie potrafiła wygenerować poprawnych, niepustych plików z dowodami (*proofs*).

Z tego powodu nie mogliśmy w pełni domknąć automatycznej weryfikacji przez **drat-trim**. Wszystkie przypadki UNSAT (gdy kolorów było za mało) zweryfikowaliśmy jednak bezpośrednio w solverze **Glucose3**. Każdy test dla mniejszego k kończył się twardym wynikiem UNSAT, co udowadnia, że generator działa poprawnie i nie przepuszcza błędnych kolorowań.